Chapter 12 Warm-Up: Recursion

Prof. Jordan C. Hanson

1 Recursion with a List

Consider the list

```
values = [7, 12, 4, 19, 3, 8, 15, 2, 11, 6, 14, 5]
```

We want a function that returns `True` only when *every* number in the list is positive. First, recall that the following method acts on a list object:

```
x = values.pop()
```

After the statement runs, what value is stored in `x`, and what has happened to `values`? In a recursive algorithm to solve this problem, each function call should examine one number, remove it, and then ask the *same question* about the smaller list. Complete the function:

```
def all_positive(values):
    # Base case: no values remain
    if ______________________:
        return ______________

    x = values.pop()

    if ______________________:
        return ______________

    return all_positive(values)
```

For

```
test = [6, 2, 9, 4]
print(all_positive(test))
```

write the value removed at each recursive call:

$$____ \rightarrow ____ \rightarrow ____ \rightarrow ____$$

This exercise is called *tracing the calls*. What is returned when the list becomes empty? Now predict the result for

```
test = [6, 2, -9, 4]
```

At what point can the recursion stop early?

2 Recursive Factorial

For a positive integer n, the factorial is

$$n! = n(n-1)(n-2)\cdots 2 \cdot 1.$$

For example,

$$6! = 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 720.$$

Factorial has a natural recursive definition:

$$n! = n(n-1)!.$$

To make the recursion stop, we also need a **base case**:

$$0! = 1 \qquad \text{and} \qquad 1! = 1.$$

Complete the recursive function:

```
def factorial(n):
    if ______________________:
        return ______________

    return ___________________
```

Let us *trace* an example. Complete the expansion:

$$\begin{aligned} 6! &= 6(5!) \\ &= 6 \cdot 5(__!) \\ &= 6 \cdot 5 \cdot 4(__!) \\ &= \cdots \\ &= _____. \end{aligned}$$

Identify the two parts of recursion:

1. What is the **base case** in `factorial()`?
2. What is the **recursive case**?
3. In each recursive call, how does the problem become smaller?

3 Challenge Problem

What should happen if a user calls `factorial(-3)`? Add one conditional that prevents the function from recursing forever on a negative input.

Key idea: A recursive function must have a stopping case and a recursive step that moves toward that stopping case.